

Continuous KNN Join Processing for Real-time Recommendation

Chong Yang[†], Xiaohui Yu^{†‡}, and Yang Liu^{†*}

[†]School of Computer Science and Technology

Shandong University, Jinan, Shandong, China, 250101

[‡]School of Information Technology

York University, Toronto, ON, Canada, M3J 1P3

yangchong.sdu@163.com, xyu@sdu.edu.cn, yliu@sdu.edu.cn

Abstract—The explosive growth of user-generated contents in social networking websites necessitates the recommendation functionality that can push to the user the content that he/she is most likely to be interested in. Such recommendation should happen in real-time as new contents become available, because “freshness” is an important consideration in people’s content-consumption behavior. Representing users and contents as feature vectors in a high-dimensional space, we can essentially cast the problem of real-time recommendations as the problem of computing the list of k nearest neighbors of each user, which we call kNN join. Given the vast volume of contents and users, the biggest challenge is how to continuously update the kNN join results as new contents arrive. Existing methods for incremental kNN join on data streams suffer from the “curse of dimensionality” and high in-memory search cost. In this paper, we present a solution that first identifies the users whose kNN’s might be affected by the newly arrived content, and then update their kNN’s respectively. We propose a new index structure named HDR-tree in order to support the efficient search of affected users. HDR-tree performs dimensionality reduction through clustering and principle component analysis (PCA) in order to improve the search effectiveness. To further reduce response time, we propose a variant of HDR-tree, called HDR*-tree, that supports more efficient but approximate solutions. The results of extensive experiments show that our methods significantly outperform baseline methods.

Keywords—real-time recommendation, high-dimensional data, k nearest neighbor join

I. INTRODUCTION

In recent years, social applications have become important tools to help people connect and share interests. With the rapid increase in the volume and velocity of user generated contents, these applications often provide recommendation functionalities to help users quickly identify interested contents (e.g., tweets from Twitter, images from Flickr, videos from YouTube). Traditionally, such recommendations are made in an offline batch fashion. However, new contents arrive at a very fast pace in a streaming fashion in these social applications. For example, on average about 2,000 images are uploaded on Flickr in a minute. It is essential to recommend to the users new contents that are of interest while the contents are still fresh, as there is a clear tendency for users of such applications to prefer newer contents. This calls for a push-based real-time

recommendation paradigm, which has two features: (1) *real-time*, meaning that the list of recommended contents (which we refer to as items in the sequel) for a user is computed and updated in real-time as data evolve; and (2) *push-based*, meaning that long-running requests can be registered and new contents of interest continuously pushed to the users.

In this paper, we are concerned with supporting push-based real-time content-based filtering, which recommends items based on their contents and the user’s profile. The choice of content-based filtering over collaborative filtering (CF) is mainly due to the fact that the “cold start” problem for CF (i.e., not having enough rating data for new items) is especially serious in the context of social applications as new items are becoming available all the time.

If we represent items and users by feature vectors in the same space (which can be obtained in a variety of ways), a sensible and robust approach to supporting content-based filtering is to recommend to the user the items that are his/her k nearest neighbors (kNN), for a given a distance metric. As there are many users to be served at the same time, we can cast the problem of computing the recommendation lists for all the users as a kNN join problem, which is defined as finding the kNN results for a set of queries on a given dataset. As the representation of items in social applications often require a large number of features, the data in our problem are of high dimensionalities. Thus, the main issue we tackle is to efficiently process high-dimensional kNN joins of the item set and the user set in a continuous manner.

Most existing kNN join algorithms do not work well for our problem. Some methods [1][2] process kNN joins on static datasets, and may incur a high cost if applied directly to streaming data due to the re-computation of the kNN join results upon every update. There do exist works that address the kNN join problem for data streams [3][4], but only for the case where the dimensionality of the dataset is low. The efficiency of these methods deteriorates as the dimensionality of datasets increases due to the curse of dimensionality [5]. To solve this problem, Yu et al. [6] consider making incremental updates to high-dimensional kNN join results, in a disk-resident setting. They propose a kNNJoin+ algorithm based on an index called the Sphere-tree to deal with the update. However, the R-tree-like structure adopted in the Sphere-tree has mediocre pruning capabilities in high-dimensional spaces. Also, distance computation in high-dimensional spaces incurs non-trivial cost. Moreover, it is very difficult for disk-based

*Corresponding author.

solutions to meet the real-time requirement in our problem due to the high I/O cost involved.

We propose a main-memory-based solution to the problem of continuous kNN join. The basic idea of our approach is to first identify the users whose kNN results can be influenced by the updated items, followed by recomputing the kNN results for this (usually small) subset of users. We introduce a new index structure called the High-dimensional R-tree (HDR-tree) to efficiently search for affected users by the updated items. HDR-tree is built on the user set and contains information of the kNN join results. We use clustering and principle component analysis (PCA) techniques for dimensionality reduction to lower the search cost and at the same time guarantee the accuracy of results.

Since in many cases, it is acceptable that new items are recommended to most of the interested users instead of exactly all, we further consider providing fast approximate solutions. The small reduction in precision is often a sacrifice we could make in return for faster response. To this end, we propose a variant of the HDR-tree named HDR*-tree to support the fast approximate retrieval of affected users. The main difference between the HDR*-tree and the HDR-tree is that the former utilizes random projection [7] for dimensionality reduction instead of PCA for the latter. HDR*-tree is demonstrated to be more efficient than HDR-tree with a reasonable accuracy lower bound. We can also control the trade-off between efficiency and accuracy easily.

In summary, we make the following contributions.

- 1) We propose the problem of real-time content-based recommendation on social data. We model it as a continuous high-dimensional kNN join problem, and present a new index structure named HDR-tree to efficiently update the kNN join results as new contents become available.
- 2) We propose another index structure named HDR*-tree to support fast approximate solutions to the kNN join problem. We analyze the cost and accuracy of using our index structures for real-time recommendation.
- 3) We conduct experiments on both real and synthetic datasets and compare our proposals with the baseline methods to demonstrate their efficiency.

The remainder of the paper is organized as follows. In Section II we present the related work. Section III describes some preliminaries. Section IV describes our index structure HDR-tree to search for affected users, and Section V presents the algorithm to update recommendation lists. Section VI describes the index structure HDR*-tree to support fast approximate solutions. Section VII presents the experiment results, and Section VIII concludes this paper.

II. RELATED WORK

There are two main approaches to recommender systems in literature: content-based filtering and collaborative filtering. Content-based filtering [8] approaches use a discrete number of features of items to recommend additional items with similar properties. Collaborative filtering [9], on the other hand, builds

a model of user behaviours and recommends items in other users' preference lists to a user with similar interests.

As social network websites become popular, real-time recommendation attracts much attention. For example, Google New [10] constructs large scale and dynamic recommender systems based on offline models. Online update problems are addressed by Agarwal et al. [11] using the Fast Online Bilinear Factor Model (FOBFM). Rendle et al. [12] investigate online matrix factorization learning methods for rating prediction. Diaz-Aviles et al. [13] address the problem of real-time top-N recommendation in social streams using a learning to rank approach. However, the above works are mainly based on collaborative filtering and machine learning methods. Due to the "cold start" problem of collaborative filtering, new items can not be recommended before they are rated by enough users. Recently, Abbar et al. [14] work on recommending the most diverse articles to users in real-time. Parallel to their works which offline construct the model, we provide efficient real-time recommendations to users by proposing a continuous k nearest neighbors join method in this study.

The kNN methods are frequently used in recommendation systems. As items are often represented by more than one hundred of features, data in content-based recommendation are of high dimensions. Indexing structures designed to cope with high-dimensional kNN query have been studied extensively. To reduce the effect of the curse of dimensionality [5] many techniques are proposed such as data approximation (e.g., VA-File [5]), one-dimensional transformation (e.g., iDistance [15]), and dimensionality reduction (e.g., Δ -tree [16]). For approximate similarity queries, locality-sensitive hashing (LSH) [17] is an efficient method which can be considered as performing probabilistic dimensionality reduction. Böhm et al. [2] first propose the kNN join problem which finds the k nearest neighbors for a set of queries in a single-run operation. MuX [2], Gorder [18] and iJoin [1] are kNN join algorithms proposed for high-dimensional datasets which mainly use nested loop techniques. However, these index structures are mostly designed for static datasets. To continuously process kNN joins for users on the item stream, we have to always recompute the kNN results for all the users, which can be very expensive.

The problems of kNN and kNN join processing on data streams are mostly studied in a low-dimensional setting. Koudas et al. [19] present the Disc technique to efficiently report the approximate answers to kNN queries. Böhm et al. [4] aim at efficient processing of exact kNN queries. They use a grid structure to index queries and use skyline techniques to monitor query results. Wee and Stephane [3] introduce the framework of progressive and approximate join algorithms on data streams. Their methods do not work well in high-dimensional datasets as the increase of dimensionality. Yu et al. [6] consider the incremental update of high-dimensional kNN joins in a disk-resident situation. They mainly focus on decreasing I/O cost, and the kNNJoin+ algorithm suffers from the problem of high-overlapping of nodes and the non-trivial cost of computing distances in high-dimensional space.

Our work is also related to the reverse kNN (RkNN) query problem, which retrieves all the objects that have the query as one of their k nearest neighbors. Korn et al. [20] propose an algorithm based on an index called Rnn-tree which precomputes for each data point its nearest neighbor. Yang et

al. [21] improve the Rnn-tree to make it efficient for both NN query and RNN queries. Stanoi et al. [22] eliminate the need for precomputing all NNs by utilizing some properties of RNN retrieval and pruning directly on a data partitioning index. Tao et al. [23] propose a TPL pruning algorithm to deal with multidimensional RkNN query. Again, all these structures do not work well on datasets with high dimensions. High-dimensional RNN and RkNN query are studied mostly for disk-resident situations [24][25]. However, they consider different situations to ours and their structures do not work well on main-memory datasets in our problem.

III. PRELIMINARIES

A. Content-based filtering

We use the popular approach of content-based filtering to make recommendations. Suppose we have a user set $U = \{u_1, \dots, u_N\}$ and an item set $I = \{i_1, \dots, i_M\}$. In content-based filtering, it is common to represent items and users by d -dimensional vectors with each dimension corresponding to a feature, and the k nearest neighbor (kNN) items for each user are recommended. In this paper, we adopt the frequently used Euclidean distance to measure the similarity between users and items [8], but the solutions can be generalized to other distance metrics as well.

As we need to find the kNN results in I for every user in U , we can perform kNN join processing to solve the problem. As shown in Definition 1, kNN join aims to find the kNN results for a set of queries on a dataset in a single-run operation [2]. There have been many existing algorithms for efficient kNN join processing in literature.

Definition 1 (KNN Join): In the d -dimensional space, given a query point set $Q = \{q_1, q_2, \dots, q_n\}$ and an object point set $O = \{o_1, o_2, \dots, o_m\}$, the kNN join returns the k nearest neighbors on O for each query in Q .

B. Real-time recommendation on social streams

A recommendation list R_j is maintained for every user u_j that contains the items that are kNN's of the user. As most users only care about the recent items in social network applications, we do not need to recommend an item any more if a considerable amount of time has passed by. We thus give every item a lifetime; the item can be a candidate for recommendation only during that time period. We use a sliding window W to monitor the item streams and only the items within the window can be recommended. While new items appear or old items expire, we need to update the recommendation lists for the users in U accordingly.

For example, in Fig. 1, every line corresponds to an item. The start of the line represents the time when the item appears in the sliding window W and the end of a line represents the time when the item expires in W . The vertical axis represents the distance between an item and u_j . Assuming that we recommend the 2NN items to every user, we will recommend $\{i_1, i_4\}$ to u_j at time 0 because they are nearest to u_j . At time t_1 , item i_2 appears in W , which is nearer to u_j than i_1 and i_4 , so we need to update R_j to $\{i_2, i_4\}$. At time t_2 , item i_2 is removed from W for exceeding its lifetime, so we delete it from R_j and push new recommendation results $\{i_1, i_4\}$ to u_j .

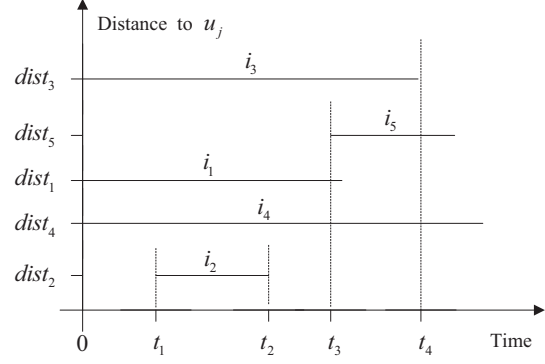


Fig. 1. Example of real-time recommendation

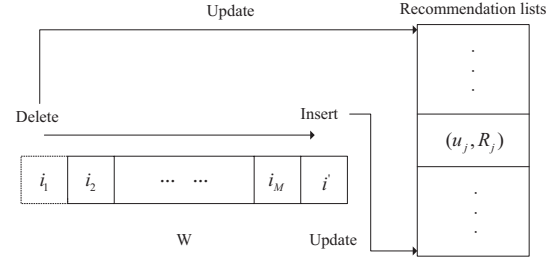


Fig. 2. Update of the join results for a sliding window

At time t_3 and t_4 , as the appearance of i_5 and the expiration of i_3 do not affect the recommendation list of u_j , we do not need to update R_j .

C. KNN join with incremental update

The main challenge is to continuously process kNN join on a sliding window of items as shown in Fig. 2. To efficiently update the join results, we use a two-phase method: the retrieval phase to find the affected users by the updated item, and the recommendation phase to update the recommendation lists. The retrieval task is actually a reverse k nearest neighbors problem [26] as presented in Definition 2.

Definition 2 (Reverse KNN): In the d dimensional space, given a query point set $Q = \{q_1, q_2, \dots, q_n\}$ and an object point set $O = \{o_1, o_2, \dots, o_m\}$, reverse kNN (RkNN) query for q_j in Q is to find all the points that contain q_j as one of their k nearest neighbors.

An updated item i' 's affected user set is equivalent to i' 's RkNN results set on U . Directly searching for RkNN results in high-dimensional space is often expensive. However, if we have the kNN items for every user in U already, then we have the following observations. For any new item, if it falls into the boundary sphere of u_j that centers at u_j with a radius equal to the distance between u_j and its current k -th NN, it will become one of the kNN's of u_j . Similarly, for any expired item, if it is within the boundary sphere of u_j , it must have been in the kNN list of u_j . So we have the following lemma, which has been exploited in previous work [6].

Lemma 1: A new (expired) item i' will affect the recommendation list of a user u_j in U , if and only if i' is in the sphere centered at u_j with radius $dknn_j$, where $dknn_j$ is the

TABLE I. SUMMARY OF NOTATIONS

I, U	item set and user set
i, u	item and user
W	the sliding window
k	the number of nearest neighbors
d, r	the dimensionality of datasets
R, R'	the kNN and RkNN result sets
f	the fanout of the tree
l	a level of the tree
L	the height of the tree
C	clusters
p, q, o, p', q'	points
P, Q, O	point sets
n, m, N, M	the size of a set
X, V	matrix
$\alpha, \beta, \theta, \eta, \varepsilon$	associated parameters of the tree
$dist$	distance
$dist_{pca}$	distance after PCA transformation
$dist_{rp}$	distance after random projection
$dknn$	distance to the k th nearest neighbor

maximum distance between u_j and its k nearest neighbors before i' 's appearance (expiration).

According to the lemma, we mean to build an index on U using the $dknn$ value, which can be then used to search for the affected users by each updated (i.e., new or expired) item. After we get the set of affected users of an updated item i' , only the recommendation list for each user u_j in the set has to be updated in the recommendation phase. If i' is a new item, we add i' into R_j and rank R_j to keep the kNN results. If i' is an expired item, we delete i' from R_j . Because the expiration of an item will cause R_j to contain less than k results, we have to recompute R_j when this happens.

Some notations used in this paper are given in Table I.

IV. HDR-TREE

As discussed in the previous section, the first phase is to identify the affected users by the updated items, which is actually an RkNN query problem on U for each updated item i' . As the data are of high dimensionality, we use the principle component analysis (PCA) technique to reduce computational cost, and we propose a new main-memory index structure called High-dimensional R-tree (HDR-tree). We build the index on U and search for the affected user set based on Lemma 1.

A. Principle component analysis

PCA [27] is an important dimensionality reduction technique. When points are transformed into lower dimensional space, the computational cost can be greatly reduced. The process of computing PCA is as follows. Let $X_{[d \times N]}$ be a point matrix, where d is the dimensionality and N is the number of points. We first compute its covariance matrix Y , and then compute the eigenvalues and eigenvectors of Y . After that, we rank the eigenvalues and choose the eigenvectors corresponding to the r largest eigenvalues to form a transformation matrix $V_{[r \times d]}$. Finally, we transform a point p to the r -dimensional space by multiplying it with V .

PCA has the following good properties [16]. Let p_1, p_2 be the d -dimensional points that are reduced to a r ($r < d$) dimensional space by PCA. Let the points after dimensionality reduction be p'_1, p'_2 . Then, if $dist(p'_1, p'_2) > c$ (c is a constant),

we have $dist(p_1, p_2) > c$. Because only some dimensions are preserved, the distance between two points in the lower dimensional space can be less than the actual distance in the original space. When a new point is added in, we can simply transform it to the lower PCA space by multiplying it with the precomputed V .

B. Index structure

The structure of HDR-tree is similar to that of the R-tree which is formed by hierarchically partitioning the dataset. One of the differences is that we are dealing with the lower dimensional data in the non-leaf node. We reduce data in the user set U into several lower dimensional spaces using PCA. Each level of the tree represents the data space with different dimensions such that the root level has the least number of dimensions and the leaf level contains the data of full dimensions. The dimensionality at each level of the tree monotonically increases from the root to the leaf nodes. The structure of HDR-tree is shown in Fig. 3.

The partitioning is done through clustering. At the root level, users in U are transformed to the d_1 -dimensional space by PCA and is partitioned into f clusters $C_1^1, C_2^1, \dots, C_f^1$. At the second level, each C_i^1 is further partitioned into sub-clusters $C_{i1}^2, C_{i2}^2, \dots, C_{if}^2$ in the d_2 -dimensional space. The process is repeated at each level l , where clusters are partitioned in the d_l -dimensional space. The users in their full dimensionality are stored in the leaf nodes.

Each cluster in the non-leaf node is represented by a tuple (*center, radius, number, maxdknn, l, ptr*), where *center* is the center of the cluster in the d_l -dimensional space, *radius* is the radius of the cluster, *number* is the number of users in the cluster, and *maxdknn* refers to the maximum distance between the users in the cluster and their k nearest neighbor items. *ptr* is the pointer to the node at the next level. In the leaf node, the actual user set U_j in the cluster is stored with no dimensionality reduction.

C. Construction

The construction of HDR-tree follows a top-down approach. We fix the fanout of the tree and partition the user set using clustering techniques at different dimensions. Here we choose the commonly used k-means algorithm as the clustering method, but other methods can also be used. If the number of users in a node is below a threshold θ , we make it a leaf node; otherwise, we make it a non-leaf node and push it into an FIFO queue for further partitioning. The procedure stops when the queue is empty.

Notice that HDR-tree is not a height-balanced tree. But it is more efficient because real world datasets are often skewed. Sparse clusters are not further partitioned which can help to reduce the number of nodes, and dense clusters are partitioned to become more fine-grained to improve the effectiveness of pruning during search. The height of the tree can be estimated as $L = \lceil \log_f N \rceil$, where f is the fanout. θ is set to a value that avoids making the actual level of tree exceed L too much. When the level of a cluster in the construction procedure is greater than L and the size is larger than θ , we continue to partition the cluster using the full dimensional user set, which means $d_l = d$ (if $l > L$).

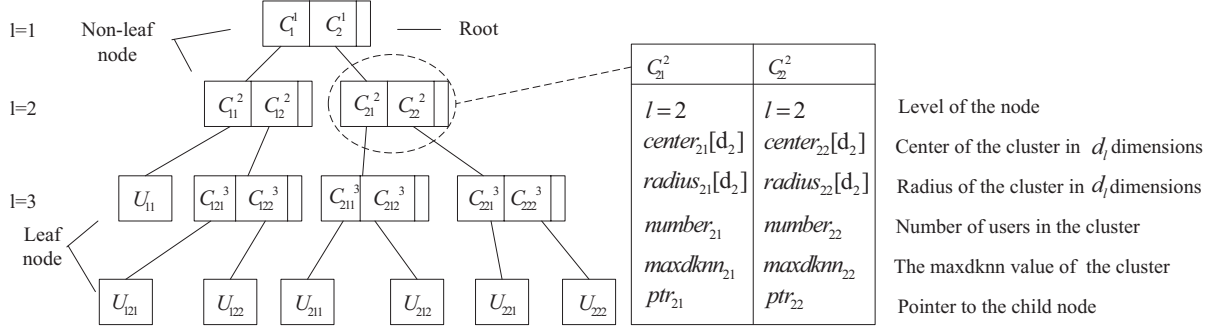


Fig. 3. An example of HDR-tree with $f = 2$. C_{ij}^l is the sub-clusters formed on the user set of C_i with d_l dimensions and $d_1 < d_2 < d_3$. The structure of a non-leaf node is shown to the right of the tree. The leaf nodes store the actual users with their $dknn$ value.

Algorithm 1 Construction Algorithm

Input: U, f, θ

Output: HDR-tree

```

1:  $C^1 = \text{new Clusters}(U, f, d_1)$ ;
2: for each  $C_j^1 \in C^1$  do
3:    $\text{maxdknn}_j^1 = \text{Max}(C_j^1.dknn)$ ;
4: end for
5:  $\text{initial root} = \text{new NonLeafNode}(C^1, \text{maxdknn}^1, 1)$ 
6:  $\text{FIFO queue queue} = \text{null}$ ;
7:  $\text{push root in queue}$ ;
8: while  $\text{queue} \neq \text{null}$  do
9:    $\text{node node} = \text{queue.get}()$ ;
10:   $C' = \text{node.clusters}$ ;
11:  for each clusters  $C_j' \in C'$  do
12:    if  $C_j'.size() < \theta$  then
13:       $LN = \text{new LeafNode}(C_j')$ ;
14:       $n.children[j] = LN$ 
15:    else
16:       $\text{level} = \text{node.l} + 1$ 
17:       $C'' = \text{new Clusters}(C_j', f, d_{\text{level}})$ ;
18:      for each  $C_j'' \in C''$  do
19:         $\text{maxdknn}_j'' = \text{Max}(C_j''.dknn)$ ;
20:      end for
21:       $NLN = \text{new NonLeafNode}(C'', \text{maxdknn}'', \text{level})$ ;
22:       $n.children[j] = NLN$ 
23:       $\text{push NLN in queue}$ ;
24:    end if
25:  end for
26: end while

```

As in our problem we assume that the user set does not change, we can store the transformation matrix and use them to map new items from the original space to the corresponding lower dimensional spaces. We use the eigenvalues $\{\gamma_1, \gamma_2, \dots, \gamma_d\}$, which are produced in the PCA algorithm and are in a decreasing order, to decide the dimensionality of each level. We first choose a value d_1 as the dimensionality of the first level. Then the dimensionality d_l of each level is chosen as the smallest value that satisfies:

$$\sum_{i=1}^{d_l} \gamma_i \geq \frac{l-1}{L-1} \left(\sum_{i=1}^d \gamma_i - \sum_{i=1}^{d_1} \gamma_i \right) + \sum_{i=1}^{d_1} \gamma_i \quad (1 < l \leq L)$$

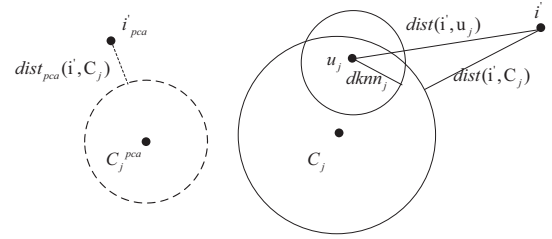


Fig. 4. Prune a node in a lower dimensional space

D. Search algorithm

We search for the affected users on HDR-tree according to Theorem 1, by which we can prune branches of the HDR-tree during the search procedure. For example, in Fig. 4, C_j^{pca} refers to a cluster of $node$ which is partitioned in the lower dimensional space. The cluster in the original space is represented by C_j . We transform an updated item i' into the same lower dimensional space to C_j^{pca} using the stored eigenmatrix. According to the property of PCA, for any user point u_j in the cluster we have

$$\text{dist}_{pca}(i', C_j) < \text{dist}(i', C_j) \leq \text{dist}(i', u_j)$$

So if we have $\text{dist}_{pca}(i', C_j) \geq \text{maxdknn}$, we can get $\text{dist}(i', u_j) > \text{maxdknn} > \text{dknn}_j$, meaning that all users in the cluster will not be affected by i' and its child nodes do not have to be scrutinized further according to Lemma 1.

Theorem 1: Suppose the distance between an item i' and a cluster C_j^l of the non-leaf node $node$ in the HDR-tree in the d_l -dimensional space is greater than the maxdknn value of C_j^l . Then we can make the following assertions: (1) If i' is a new item, no users in the cluster will contain i' as one of its k nearest neighbors after i' is inserted. (2) If i' is an expired item, no users in the cluster have contained i' as one of their k nearest neighbors.

We make a depth-first search on the HDR-tree. The pseudocode of searching for affected users of an item is shown in Algorithm 2. The starting node is initialized as the *root* of the tree. Lines 1 to 6 deal with the leaf nodes by direct computation and comparison. In Lines 8 to 14, we prune those non-leaf nodes which satisfy the conditions set out in Theorem 1 and send the rest of the nodes to the next round of search. The whole procedure can prune all users that cannot be

Algorithm 2 HDRSearch

Input: node $node$, updated item i' **Output:** affected user set R'

```

1: if  $node$  is a leaf node then
2:   for each users  $u_j$  in  $node$  do
3:     if  $dist(i', u_j) \leq u_j.dknn$  then
4:       add  $u_j$  in  $R'$ .
5:     end if
6:   end for
7: else
8:    $C' = node.clusters$ 
9:   for each clusters  $C'_j \in C'$  do
10:    if  $dist_{node.dimension}(i', C'_j) < C'_j.maxdknn$  then
11:      HDRSearch( $node.children[j], i'$ );
12:    end if
13:  end for
14: end if
15: return  $R'$ 

```

affected by the updated item. Although the information loss in dimensionality reduction causes more nodes to be visited than that in the original space, the computational cost of searching for influenced users on HDR-tree is greatly reduced because of the good properties of PCA.

V. UPDATING RECOMMENDATION LISTS

Once the affected users are identified, we can update the kNN list of those users. In this section, we present the algorithm for updating the list and analyze the cost of computation.

A. Update algorithm

After the retrieval part of finding the affected user set R' , we only need to update the recommendation lists of users in R' . We can see the procedure of update in Algorithm 3. The HDR-tree also needs to be updated because the $maxdknn$ of the clusters containing the users in R' must be recomputed. *AdjustMaxdknn* is the method to adjust the $maxdknn$ of corresponding clusters. It can be completed by utilizing a recursive scan on HDR-tree to find the nodes containing affected users in R' and to adjust the $maxdknn$ value of all the clusters in the searching path. Notice that the expiration of an item will cause the number of the user's kNN results to become less than k . We have to recompute its kNN results as the new recommendation list by computing the distances between the user to all items in the sliding window.

B. Cost analysis

As we consider all data in main memory, the dominant cost is the computational cost instead of I/O cost as in previous disk-based solutions. Updating recommendation lists mainly consists of three parts of cost.

The first is $cost_{RKNN}$, the cost of searching for affected users on HDR-tree. In Algorithm 2, we use α to refer to the average probability to search the sub-tree of a node. The fanout of tree is f . The height of the HDR-tree can be estimated as $L = \lceil \log_f N \rceil$. Then the total number of computed non-leaf nodes can be estimated as $((f\alpha)^{L-1} - 1)/(\alpha - 1)$.

Algorithm 3 Update Algorithm

Input: updated item i' **Output:** updated R

```

1:  $R' = \text{HDRSearch}(root, i')$  on HDR-tree;
2: for each  $u_j$  in  $R'$  do
3:   update the recommendation list of  $u_j$ ;
4:   if  $R_j.size() < k$  then
5:     compute the kNN results for  $u_j$  as the new  $R_j$ ;
6:   end if
7:   update  $dknn_j$ ;
8: end for
9: AdjustMaxdknn on HDR-tree;

```

The cost of the computation to non-leaf nodes is $((f\alpha)^{L-1} - 1)d_{level}/(\alpha - 1)$, where d_{level} is the average dimensionality of each level. The cost of distance computation in leaf nodes is $(f\alpha)^L d$ where d is the original dimensionality. So we can get the theorem below.

Theorem 2: The cost of searching for affected users using the HDR-tree can be estimated as

$$cost_{RKNN} = ((f\alpha)^{L-1} - 1)d_{level}/(\alpha - 1) + (f\alpha)^L d$$

The second part is the cost of update operation on the kNN join results, $cost_{KNN}$. It differs on whether the updated item is a new or an expired item. We assume that the size of the sliding window is approximately fixed, meaning the average number of appearance equals to the number of expiration in a long-run processing. The average number of affected users for one update can be estimated as $|U|k/|W|$, where $|W|$ is the size of the sliding window. If the update item is a new item, we need to insert the item into the users' kNN lists. The cost is $O(|U|k^2/|W|)$. If the update item is an expired item, we need to recompute the kNN results for the affected users. The cost is $O(|W| * d * |U|k/|W|) = O(|U|dk)$. At last, we can estimate the cost of update kNN join results as $cost_{KNN} = 0.5 * O(|U|k^2/|W|) + 0.5 * O(|U|dk)$.

The last part is the maintenance cost of HDR-tree, $cost_{MA}$, which can be done in one traverse of the tree for all the affected users. so we can estimate $cost_{MA} = O(d_{level} * L * f * |U|k/|W|)$.

VI. HDR*-TREE FOR APPROXIMATE PROCESSING

From the cost analysis we can see that how well the distance is preserved in lower dimensional spaces affects the efficiency of search on HDR-tree. Although PCA technique preserves the most important components of the points, it is not necessarily the best method to preserve the distances in lower dimensional spaces. In addition, it often suffices to return approximate answers to the kNN joins in the sense that it is acceptable for some users to be missing from the affected user list by a new item with a small probability, in return for faster processing.

We thus propose a new index structure named HDR*-tree to approximately search for affected users. We improve HDR-tree by using another dimensionality reduction method called random projection [7], which uses a random projection matrix to project data to a lower dimensional subspace. The

main benefit of random projection is that it can approximately preserve the distance between points in lower dimensional space [28]. The clustering effect on lower dimensional space of random projection is also better than that of using PCA [29]. Furthermore, the transformation matrix of random projection can be acquired much easier than PCA without great computation cost.

A. Random projection

In random projection, the original d dimensional data can be projected into a r ($r \ll d$) dimensional subspace through a $r \times d$ random matrix V whose columns have unit lengths. The matrix notation of a matrix $X_{[d \times n]}$ projected to a lower dimensional subspace is

$$X'_{[r \times n]} = V_{[r \times d]} X_{[d \times n]}$$

where n is the total number of points. The main idea of random projection comes from the Johnson-Lindenstrauss Lemma [7].

There are many choices for the random matrix, and one of the most frequently-used transformation matrix is suggested by Achlioptas [7]. The elements of the random matrix are drawn from the following distribution:

$$v_{i,j} = \begin{cases} +\sqrt{3} & \text{with probability of } 1/6 \\ 0 & \text{with probability of } 2/3 \\ -\sqrt{3} & \text{with probability of } 1/6 \end{cases}$$

Let V be a $r \times d$ random matrix generated from the above distribution. Suppose that X is a $d \times n$ matrix of n points. For given constants $\varepsilon, \beta > 0$, we choose an integer g such that g satisfies

$$g \geq g_0 \approx \frac{4 + 2\beta}{\varepsilon^2/2 - \varepsilon^3/3} \log n.$$

Then we can get the projected matrix $X'_{[r \times n]} = \frac{1}{\sqrt{g}} V_{[r \times d]} X_{[d \times n]}$. We have the following results regarding the distances between projected points.

Theorem 3: For any points p, q in the matrix $X_{[d \times n]}$, we have

$$(1 - \varepsilon) \text{dist}(p, q) \leq \text{dist}(p', q') \leq (1 + \varepsilon) \text{dist}(p, q)$$

with probability of at least $\eta = 1 - n^{-\beta}$, where p' and q' are the projected points of p and q in $X'_{[r \times n]}$ respectively.

We call ε the *distortion value*, and we use ε and η as the parameters of the HDR*-tree.

B. Index structure

The structure of the HDR*-tree similar to that of the HDR-tree, except that we use random projection instead of PCA to transform user profiles in U into different dimensional spaces. Again, the dimensionality increases from the root to the leaf level. We add ε into the representation of the clusters of the non-leaf nodes at each level, so that each cluster in a non-leaf node is represented by a tuple (*center, radius, number, maxdknn, l, ε_l , ptr*). Similar to the HDR-tree, the leaf node contains the full user representation with no dimensionality reduction.

To determine the dimensionality of each level, we fix η of the random projection procedure, and estimate the smallest distortion value ε_{min} that can be achieved. We choose a value ε_{max} as the distortion value of the root level of the tree. The distortion value of every level of the tree is between these two extremes. To be more specific, the distortion value at each level l is estimated as:

$$\varepsilon_l = \varepsilon_{max} - \frac{l-1}{L-1} * (\varepsilon_{max} - \varepsilon_{min})$$

Then the number of dimensions at each level is chosen to be the smallest value that can satisfy the parameters η and ε_l . In our setting, we choose $\varepsilon_{min} = 0.25$ and $\varepsilon_{max} = 1.00$. Achlioptas et al. [7] give the limit condition of the parameters in the worst cases. But it has been shown that random projection has much better performance in practice [28].

C. Search algorithm

We have the following result on the HDR*-tree.

Theorem 4: If the distance between an item i' and a cluster C_j^l of the non-leaf node *node* in the HDR*-tree in the d_l dimensional space satisfies the following condition,

$$\text{dist}_{rp}(i', C_j^l) > (1 + \varepsilon_l) \text{maxdknn},$$

where *maxdknn* is the *maxdknn* value of C_j^l , then we can assert that (1) if i' is a new item, then no user in the cluster will have i' as one of its k nearest neighbors after i' is inserted with a probability of at least η ; or (2) if i' is an expired item, then no user in the cluster has had i' as one of his/her k nearest neighbors with a probability of at least η .

Proof: According to Theorem 3, we have $\text{dist}_{rp}(i', C_j^l) < (1 + \varepsilon_l) \text{dist}(i', C_j)$, where C_j is the cluster in the original space, with probability of at least η . So if the condition $\text{dist}_{rp}(i', C_j^l) > (1 + \varepsilon_l) \text{maxdknn}$ holds, then with probability of at least η , we have $\text{dist}(i', C_j) > \text{maxdknn}$. For any user u_j in the cluster, we have $\text{dist}(i', u_j) \geq \text{dist}(i', C_j) > \text{maxdknn} > \text{dknn}_j$ with probability of at least η , which means that i' will not be one of u_j 's k nearest neighbors if i' is a new item, or i' has not been in u_j 's k NN list if i' is an expired item. ■

The search algorithm on HDR*-tree prunes clusters based on Theorem 4. The whole procedure is similar to the algorithm for HDR-tree except that we search the sub-tree of the clusters in the node only when $\text{dist}(i', C_j^l) \leq (1 + \varepsilon_l) \text{maxdknn}$. We keep the random matrix used in random projection, and utilize it in the update process so that all points are projected into the same sub-spaces.

D. Accuracy analysis

Random projection is performed at each non-leaf level of the HDR*-tree to reduce dimensionality. According to Theorem 3, there may exist users that fall out of the distortion interval with a probability of less than $1 - \eta$. Therefore, when we prune clusters according to Theorem 4, a cluster that contain affected users may be pruned by mistake, which causes the incompleteness of the search result (false negatives).

We use *recall* to measure the accuracy of the search for affected users using HDR*-tree.

Definition 3: Let the complete set of affected users by the updated item i' be R' , and the set of affected users obtained by searching HDR*-tree be R^* . Recall is defined as

$$Recall = \frac{|R^* \cap R'|}{|R'|}$$

Theorem 5: The lower bound of the recall when using HDR*-tree to search for influenced users is

$$(1 - c(1 - \eta))^{L-1}$$

where η is the probability in random projection, L is the height of the tree, and c is a constant ($c < 1$).

Proof: According to Theorem 3, a point falls outside of the distortion region of another point with a probability less than $1 - \eta$. So a user u_j in R' is not contained in R^* in each level with probability less than $c(1 - \eta)$, where c is a constant less than 1 because the radius of the boundary sphere of u_j is no larger than $maxdknn$. A user point in R' is not contained in R^* in the whole tree with the probability less than $1 - (1 - c(1 - \eta))^{L-1}$. The number of points in $|R^* \cap R'|$ is more than $(1 - c(1 - \eta))^{L-1}R'$. So the recall is greater than $(1 - c(1 - \eta))^{L-1}$. ■

The cost analysis for the HDR*-tree is similar to that for the HDR-tree. Random projection performs better than the latter in two aspects. First, the computation of random projection costs less than linear dimensionality reduction (PCA). The time complexity of random projection is only $O(ndr)$ where n is the number of points, d is the original dimensionality, and r is the reduced dimensionality. Second, random projection performs better in clustering in lower dimensionality which can be seen in experiments of [28][29]. This means more unnecessary points may be pruned and thus HDR*-tree has less computation cost.

VII. EXPERIMENTS

We conduct experiments to validate the efficiency of using our structures of incremental kNN joins for real-time recommendation. We compare our strategies based on HDR-tree and HDR*-tree with two baseline methods found in the literature that could be used for searching affected users:

- 1) *NaiveRKNN*: It is a naive method to search for affected users without using any indexes by computing the distances between the updated item and all the user profiles in U . For each user, it decides whether its kNN list should be updated.
- 2) *KNNJoin+*: It is an existing algorithm for high-dimensional kNN join that allows for incremental update. It uses Sphere-tree [6] to find the RkNN results of the updated items. It is a disk-based solution, but is adapted to the main-memory setting in our experiments.

Our experiments are conducted on both real world and synthetic datasets. The real world dataset comes from the 128-dimensional wavelet texture data from NUS-WIDE Image Data Set [30], which contains 269,648 records from Flickr. We also generate synthetic datasets with different dimensionalities and different number of features for more controlled studies using methods similar to those used in the work of Chakrabarti et al.

TABLE II. DEFAULT VALUES

parameter	default value
window size	200,000
size of user set	50,000
k	10
dimension	128
η	0.95

[27]. To mimic the real application where the items appear in streams, we create “streaming” versions of these datasets with the window size set at 200,000. We randomly choose 50,000 records from the dataset as the user set. In the case of image dataset, this can be understood as representing the users by the images they liked in the past. Some of the default values used in the experiments are shown in Table II. The efficiency is measured by the average elapsed time and number of distance computations on 100 updates. All experiments¹ are conducted using Java language on a Dell machine with an Intel Core i5-2400 CPU, 8G RAM, running Windows7 x64 OS.

A. Effect of Parameters

1) *Varying the fanout*: We study the effect of the fanout of our index structures on the performance. Let the fanout range from 3 to 40, and the elapsed time as well as the number of distance computations in Fig. 5. The number of distance computations increases when the fanout gets larger, because a larger fanout leads to a smaller height of the tree and a large size of the nodes, which negatively affects the pruning ability. However, from Fig. 5(a) we can see that the elapsed time first decreases before the fanout reaches 5 and then increases after that. The reason comes from the computation with the nodes which decreases with increased fanout. When the fanout is greater than 5, the increase in the computation time with the user profiles in the leaf nodes becomes dominant so that the curves go upward after that. In summary, for a given dataset, there is an optimal choice of the fanout that gives the the least elapsed time of the proposed index structures. In the rest of the experiments, we choose 5 as the default fanout value. The parameters of the other structures are also set to their optimal values.

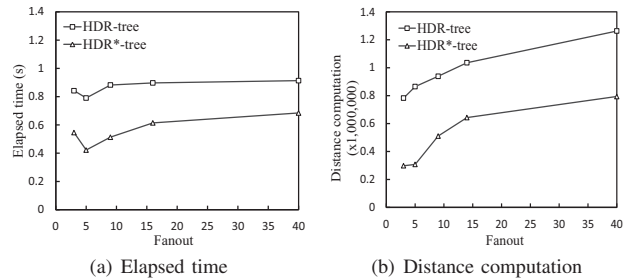


Fig. 5. Performance w.r.t fanout

2) *Varying η for HDR*-tree*: η is the dominant parameter in constructing the HDR*-tree. We vary η from 0.55 to 0.95, and the result on the elapsed time is shown in Fig. 6(a). The cost of searching HDR*-tree to find affected users grows as η increases because a larger η causes the dimensionality of each level and hence the search cost to be higher. On the flip side,

¹<https://github.com/chongzi1990/continuouskNNjoin.git>

we can see from Fig. 6(b) that a larger η makes the kNN join results more accurate, as shown in Theorem 4. In the rest of the experiments, we choose $\eta = 0.95$ unless otherwise noted in order to ensure a high level of accuracy of the result. It is worth noting that the performance of HDR*-tree still is better than HDR-tree when η is large (Fig. 7).

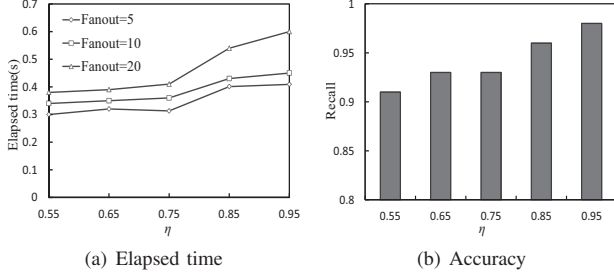


Fig. 6. Performance w.r.t η for HDR*-tree

B. Comparison with existing methods

1) *Varying the number of updated items:* We first compare our methods with the two baseline methods in terms of the total elapsed time when we vary the number of updated items from 100 to 600, and the result is shown in Fig. 7. The cost of all indexes increases when the number of updates becomes larger, as each update requires a search for affected users. The cost is roughly linear with respect to the number of updated items since each update requires an independent search. Apparently both of our methods based on the HDR-tree and HDR*-tree run significantly faster than the baseline methods.

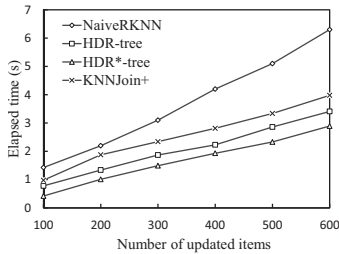


Fig. 7. Performance w.r.t the number of updated items

2) *Varying the size of user set:* We test the efficiency on different sizes of the user set. We can see from Fig. 8 that the HDR-tree-based method costs less than 1/2 time of the NaiveRKNN methods and HDR*-tree costs less than 1/3 time. Our methods also perform better than Sphere-tree in the kNNJoin+ algorithm for the following reasons. First, our indexes have a better pruning power and thus incur less distance computations (which can be seen in Fig. 8(b)) due to the use of the clustering technique, as discussed in Section IV. Second, the dimensionality reduction technique helps to reduce the cost of computation compared to the kNNJoin+ algorithm where distance computation is done in the original higher dimensional space. We can also see that the HDR*-tree index performs better than HDR-tree. The reason is that HDR*-tree can prune more points because random projection can better preserve the distance in lower dimensional space. The PCA technique reduces a large number of dimensions but does

not give consideration to keeping the distance relationships between points, and therefore more nodes are likely to be visited in the HDR-tree than in the HDR*-tree. Admittedly, the increase in performance comes at the price of accuracy.

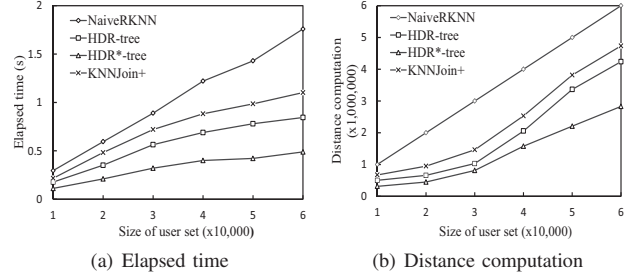


Fig. 8. Performance w.r.t the size of the user set

3) *Varying $|W|$ and k :* Figure 9(a) shows that with the increase of the size of sliding window $|W|$, the elapsed time of all the methods increases. The reason is that the recalculation cost become larger when we an item expires. Also, from the cost analysis we can see that the cost of searching for affected users decreases as $|W|$ rises because the average number of affected users decreases. Clearly, the increase of recalculation cost is dominant, which makes the curves go upward. In Fig. 9(b), we can see that the elapsed time of all the methods increases when the number of items recommended to each user k increases. The reason is that a larger k means a larger $maxdknn$ value and more users affected by an update item. We can see that recommending the 25NN items to each user costs 30% more elapsed time for both of our methods compared to recommending the 1NN item.

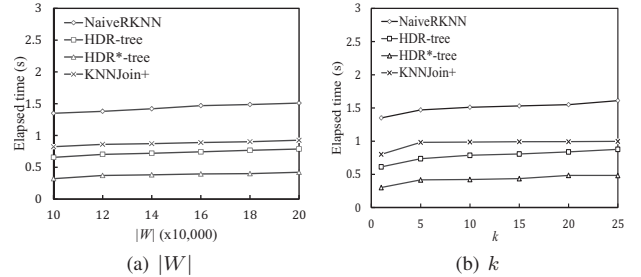


Fig. 9. Performance w.r.t $|W|$ and k

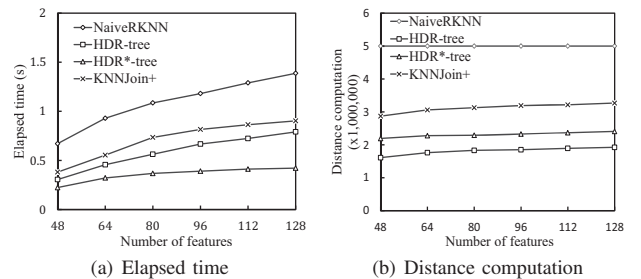


Fig. 10. Performance w.r.t the number of features

4) *Varying the number of features:* We then conduct experiments on the generated datasets with different dimensionalities, using items with different numbers of features. Figure 10(a)

shows that the elapsed time increases as the number of features grows because higher dimensionality increases the time of distance computation. We can see that the cost of our structures increases at a slower rate as the number of features increases, which means that our indexes are more robust than the rest of methods. Figure 10(b) shows the numbers of distance computations. NaiveRKNN computes distances with all the users in all the datasets so that its number of distance computations stays fixed. The number of distance computations of the tree structures increases when the number of features gets larger. The reason is that there is more overlapping between nodes in higher dimensional space, which causes the drop of pruning ability of the tree structures. Still, both HDR-tree and HDR*-tree perform better than existing methods for all the item sets with different numbers of features.

VIII. CONCLUSIONS

We have addressed the problem of continuous kNN join processing for real-time content-based recommendation. As items in social applications are often represented by feature vectors of hundreds of dimensions, the main issue is to efficiently maintain the high-dimensional kNN join results as data evolve. The basic idea of our approach is to first identify the users whose kNN results can be influenced by the update, followed by recomputing the kNN results for this (usually small) subset of users. We propose a new main-memory index structure called HDR-tree on the user set to efficiently locate the affected users by the updated items. Our index structure combines clustering and PCA technique, and is efficient for high-dimensional search. Furthermore, we propose another index HDR*-tree for the approximate search of affected users using random projection as the tool for dimensionality reduction. It runs faster than HDR-tree at the expense of slight inaccuracy (with a reasonable lower bound). The experiments on both the real and synthetic datasets have demonstrated the superiority of our proposals to existing methods for real-time recommendation. Our plan for next stage is to make a distributed implementation of the structure to further reduce elapsed time.

IX. ACKNOWLEDGEMENT

This work was supported in part by the National Natural Science Foundation of China Grant (61272092), the Shandong Provincial Natural Science Foundation Grant (ZR2012FZ004), the Science and Technology Development Program of Shandong Province (2014GGE27178), the 973 Program (2015CB352500), the Independent Innovation Foundation of Shandong University (2012ZD012), the Taishan Scholars Program, and NSERC Discovery Grants.

REFERENCES

- [1] C. Yu, B. Cui, S. Wang, and J. Su, "Efficient index-based knn join processing for high-dimensional data," *Information and Software Technology*, vol. 49, no. 4, pp. 332–344, 2007.
- [2] C. Böhm and F. Krebs, "The k-nearest neighbour join: Turbo charging the kdd process," *KAIS*, vol. 6, no. 6, pp. 728–749, 2004.
- [3] W. H. Tok and S. Bressan, "Progressive and approximate join algorithms on data streams," in *Advanced Query Processing*, 2013, pp. 157–185.
- [4] C. Böhm, B. Ooi, C. Plant, and Y. Yan, "Efficiently processing continuous k-nn queries on data streams," in *ICDE*, 2007, pp. 156–165.
- [5] R. Weber, H. Schek, and S. Blott, "A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces," in *VLDB*, 1998, pp. 194–205.
- [6] C. Yu, R. Zhang, Y. Huang, and H. Xiong, "High-dimensional knn joins with incremental updates," *Geoinformatica*, vol. 14, no. 1, pp. 55–82, 2010.
- [7] D. Achlioptas, "Database-friendly random projections," in *SIGMOD*, 2001, pp. 274–281.
- [8] M. J. Pazzani and D. Billsus, "Content-based recommendation systems," in *The adaptive web*, 2007, pp. 325–341.
- [9] J. S. Breese, D. Heckerman, and C. Kadie, "Empirical analysis of predictive algorithms for collaborative filtering," in *UAI*, 1998, pp. 43–52.
- [10] A. S. Das, M. Datar, A. Garg, and S. Rajaram, "Google news personalization: scalable online collaborative filtering," in *WWW*, 2007, pp. 271–280.
- [11] D. Agarwal, B.-C. Chen, and P. Elango, "Fast online learning through offline initialization for time-sensitive recommendation," in *KDD*, 2010, pp. 703–712.
- [12] S. Rendle and L. Schmidt-Thieme, "Online-updating regularized kernel matrix factorization models for large-scale recommender systems," in *RecSys*, 2008, pp. 251–258.
- [13] E. Diaz-Aviles, L. Drumond, L. Schmidt-Thieme, and W. Nejdl, "Real-time top-n recommendation in social streams," in *RecSys*, 2012, pp. 59–66.
- [14] S. Abbar, S. Amer-Yahia, P. Indyk, and S. Mahabadi, "Real-time recommendation of diverse related articles," in *WWW*, 2013, pp. 1–12.
- [15] H. Jagadish, B. Ooi, K. Tan, C. Yu, and R. Zhang, "idistance: An adaptive b+-tree based indexing method for nearest neighbor search," *TODS*, vol. 30, no. 2, pp. 364–397, 2005.
- [16] B. Cui, B. Ooi, J. Su, and K. Tan, "Contorting high dimensional data for efficient main memory knn processing," in *SIGMOD*, vol. 9, no. 12, 2003, pp. 479–490.
- [17] A. Gionis, P. Indyk, R. Motwani *et al.*, "Similarity search in high dimensions via hashing," in *VLDB*, vol. 99, 1999, pp. 518–529.
- [18] C. Xia, H. Lu, B. C. Ooi, and J. Hu, "Gorder: an efficient method for knn join processing," in *VLDB*, vol. 30, 2004, pp. 756–767.
- [19] N. Koudas, B. Ooi, K. Tan, and R. Zhang, "Approximate nn queries on streams with guaranteed error/performance bounds," in *VLDB*, vol. 30, 2004, pp. 804–815.
- [20] F. Korn and S. Muthukrishnan, "Influence sets based on reverse nearest neighbor queries," in *SIGMOD*, vol. 29, no. 2, 2000, pp. 201–212.
- [21] C. Yang and K. Lin, "An index structure for efficient reverse nearest neighbor queries," in *ICDE*, 2001, pp. 485–492.
- [22] I. Stanoi, D. Agrawal, and A. Abbadi, "Reverse nearest neighbor queries for dynamic databases," in *SIGMOD*, 2000, pp. 44–53.
- [23] Y. Tao, D. Papadias, X. Lian, and X. Xiao, "Multidimensional reverse knn search," *VLDB*, vol. 16, no. 3, pp. 293–316, 2007.
- [24] A. Singh, H. Ferhatosmanoglu, and A. Ş. Tosun, "High dimensional reverse nearest neighbor queries," in *CIKM*, 2003, pp. 91–98.
- [25] M. Zhang and R. Alhajj, "Effectiveness of naq-tree in handling reverse nearest-neighbor queries in high-dimensional metric space," *KAIS*, vol. 31, pp. 307–343, 2012.
- [26] I. Stanoi, M. Riedewald *et al.*, "Discovery of influence sets in frequently updated databases," in *VLDB*, 2001, pp. 99–108.
- [27] K. Chakrabarti and S. Mehrotra, "Local dimensionality reduction: A new approach to indexing high dimensional spaces," in *VLDB*, 2000, pp. 89–100.
- [28] E. Bingham and H. Mannila, "Random projection in dimensionality reduction: applications to image and text data," in *KDD*, 2001, pp. 245–250.
- [29] S. Dasgupta, "Experiments with random projection," in *UAI*, 2000, pp. 143–151.
- [30] T.-S. Chua, J. Tang, R. Hong, H. Li, Z. Luo, and Y.-T. Zheng, "Nus-wide: A real-world web image database from national university of singapore," in *ACM International Conference on Image and Video Retrieval*, 2009.